Documentação Técnica - Atividade 2

Gabriel Felipe F. de Souza 2669480 e Thiago André Mattos Málaga 2669641

Histórias de Usuário

História de Usuário 1: Reservar horário de monitoria
História de usuário: Como aluno, eu quero visualizar os horários livres e reservar um atendimento de 30 minutos em uma disciplina para garantir que serei atendido sem conflitos com outros colegas.

Critérios de Aceitação:

- O sistema deve listar as disciplinas que possuem monitoria disponível.
- O sistema deve exibir os horários livres (não reservados) em blocos de 30 minutos para a disciplina escolhida.
- Ao confirmar a reserva, o sistema deve bloquear aquele horário para outros alunos, caso o atendimento seja individual.
- O sistema deve permitir o cadastro de múltiplos horários de atendimento semanais pelo monitor.

Trecho justificador: "A ideia é que o aluno escolha a disciplina, veja os horários livres e reserve um atendimento. Cada atendimento dura 30 minutos..."

Prioridade: Alta, é a base do programa e a necessidade principal dos alunos.

História de Usuário 2: Cadastrar disponibilidade
História de usuário: Como monitor, eu quero cadastrar meus horários de disponibilidade no sistema para que os alunos possam visualizar quando estarei livre para agendamentos.

Critérios de Aceitação:

- O monitor deve poder selecionar os dias da semana e horários em que estará disponível para a sua disciplina e o máximo de atendimentos semanais por aluno.
- O sistema deve dividir automaticamente os horários em intervalos de 30 minutos.

- O monitor deve poder visualizar a sua própria grade de horários cadastrados e os que foram reservados por alunos.
- O sistema deve guardar os horários e mostrá-los atualizados quando aberto.

Trecho justificador: "No início do semestre, cada monitor informa os horários em que pode atender..."

Prioridade: Alta, sem disponibilidade não há como reservar vagas.

História de Usuário 3: Cancelar horário de atendimento
História de usuário: Como monitor, eu quero cancelar um horário previamente disponibilizado para lidar com imprevistos acadêmicos, como a realização de provas.

Critérios de Aceitação:

- O monitor deve conseguir selecionar um horário específico no sistema e marcá-lo como cancelado.
- O sistema deve informar imediatamente o horário cancelado da lista de horários livres visíveis para os alunos.
- Se o horário já estiver reservado por um aluno, o cancelamento deve avisá-lo.
- O monitor deve ser obrigado a fornecer uma justificativa (motivo do cancelamento) ao cancelar um horário que já possui aluno agendado.

Trecho justificador: "...mas às vezes precisa cancelar por causa de prova."

Prioridade: Média, por mais que não seja estritamente necessária, de vez em quando acontece imprevistos.

Projeto inicial

História de Usuário Escolhida: Reservar horário de monitoria.

História de usuário: Como aluno, eu quero visualizar os horários livres e reservar um atendimento de 30 minutos em uma disciplina para garantir que serei atendido sem conflitos com outros colegas.

Descrição da implementação - solução:

A implementação terá as classes:

1 - ServicoMonitoria:
Descrição: Implementa as regras para listagem e reserva de horários.
Atributos: Sem atributos.
Responsabilidades: Consultar disponibilidade e realizar reservas, impedindo operações inválidas.

2 - Monitor:
Descrição: Representa o monitor de uma disciplina.
Atributos: *nome(str).*

3 - Aluno:
Descrição: Representa um aluno do sistema.
Atributos: *nome(str)* e *identificador(str)*.

4 - Reserva:
Descrição: É a associação entre um aluno e um horário reservado.
Atributos: *aluno(Aluno)* e um *horário(HorarioMonitoria).*

5 - HorarioMonitoria:
Descrição: Representa um horário de atendimento de 30 minutos. Atributos: Tem atributos básicos como *dia da semana(str), hora_inicio(time), tipo(str) [individual ou grupo], capacidade(int), reservas(lista - Reserva). Também outros atributos auxiliares como hora_fim(time), status_reserva(str)*
[livre, com vagas, lotado], esta_disponível retorna true se tiver capacidade. vagas_disponiveis que retorna o número de vagas disponíveis. Responsabilidades: calcular o horário de término, verificar a disponibilidade, informar o status da reserva (livre, com vagas ou lotado) e calcular a quantidade de vagas disponíveis.

6 - Disciplina:
Descrição: Representa uma disciplina.
Atributos: nome(str), monitor(Monitor), horários(lista - HorarioMonitoria) dos horários pré cadastrados.
Responsabilidades: armazenar os horários de monitoria e permitir o cadastro de novos horários, evitando duplicidades.

Cada classe tenta resolver parte do problema, tem seus métodos especiais, getters e setters e métodos específicos de cada solução.

Funcionamento Geral:

O monitor irá cadastrar os horários de atendimento relativos à disciplina. O aluno consulta os horários disponíveis e seleciona aquele que deseja reservar. O ServiçoMonitoria verifica se o horário pertence à disciplina e se possui vagas. Caso a reserva seja válida, uma instância de Reserva é criada e associada ao horário, atualizando sua disponibilidade.

Nos atendimentos individuais, cada horário permite apenas uma reserva. Nos atendimentos em grupo, a quantidade de reservas é limitada pela capacidade definida para o horário.

Testes Automatizados:

A verificação do funcionamento do sistema é realizada por testes na classe TestMonitoria (testesgreen.py), executados via unittest assim como no enunciado da atividade. Os testes estão mapeados diretamente aos 4 critérios de aceitação da História de Usuário escolhida, e separados em testes de horário e testes de reserva:

- Critério 1: O sistema deve listar as disciplinas que possuem monitoria disponível
    - Casos de Sucesso: test_reserva_fisica, test_reserva_calculo e test_reserva_poo validam a criação, listagem e vinculação correta das disciplinas com seus respectivos monitores.
    - Caso de Falha/Exceção: test_impedir_reserva_horario_de_outra_disciplina valida que o sistema lança exceção ao tentar reservar um horário em uma disciplina à qual o horário não pertence.

- Critério 2: O sistema deve exibir os horários livres em blocos de 30 minutos para a disciplina escolhida
    - Casos de Sucesso: test_horario_tem_30_minutos e test_listar_horarios_disponiveis garante que a propriedade de término calcula exatamente o intervalo fixo de 30 minutos e confirma que os horários livres ativos são listados, respectivamente.
    - Caso de Falha: test_listar_horarios_nao_mostra_horario_lotado valida que horários que atingem sua capacidade máxima não são mostrados.

- Critério 3: Ao confirmar a reserva, o sistema deve bloquear aquele horário para outros alunos (se individual) ou atualizar as vagas restantes (se em grupo)
    - Casos de Sucesso (Individual): test_reserva_fisica valida a alteração do status para "lotado" após uma reserva.

    - Casos de Sucesso (Grupo): test_reserva_em_grupo e test_permitir_varios_alunos_em_horario_de_grupo validam a inclusão de múltiplos alunos até o limite definido.
    - Casos de Falha/Exceção: test_impedir_reserva_em_horario_lotado, test_impedir_reserva_quando_grupo_atingir_capacidade e test_impedir_aluno_reservar_duas_vezes validam os casos em que o aluno não podem fazer reservas.
- Critério 4: O sistema deve permitir o cadastro de múltiplos horários de atendimento semanais pelo monitor
    - Caso de Sucesso: test_permitir_varios_alunos_em_horario_de_grupo valida o cadastro de múltiplos horários na disciplina.
    - Caso de Falha/Exceção: test_impedir_horario_duplicado verifica se o monitor pode cadastrar um horário idêntico (mesmo dia e hora) para a mesma disciplina.

História de Usuário 4: Configurar atendimento em grupo
História de usuário: Como monitor, eu quero configurar um horário específico para aceitar múltiplos alunos simultaneamente para otimizar o tempo quando várias pessoas têm a mesma dúvida.

Critérios de Aceitação:

- O monitor deve ter a opção de marcar um horário de 30 minutos como "atendimento em grupo".
- O monitor deve poder definir a quantidade máxima de alunos para esse horário específico.
- O sistema deve permitir que diferentes alunos reservem o mesmo horário em grupo até que o limite de vagas estipulado seja atingido.
- O sistema deve exibir o número de vagas restantes (ex: "3 de 5 vagas disponíveis") para os alunos ao visualizar um horário de atendimento em grupo.

Trecho justificador: "...alguns monitores preferem atender em grupo, quando várias pessoas têm a mesma dúvida."

Prioridade: Média, depende da preferência de cada monitor.

História de Usuário 5: Receber notificação de cancelamento
Descrição: Como aluno, eu quero descobrir com antecedência quando meu atendimento for cancelado pelo monitor para evitar que eu vá até a sala desnecessariamente.

Critérios de Aceitação:

- O aluno deve receber um aviso (ex: na tela do sistema) informando que sua reserva foi cancelada pelo monitor.
- O status do agendamento na visão do aluno deve ser alterado de “Reservado” para "Cancelado".
- O sistema deve permitir que o aluno retorne facilmente à lista de horários daquela disciplina para tentar marcar uma nova reserva.
- O sistema deve registrar a data e o horário em que o cancelamento ocorreu no histórico de agendamentos do aluno.

Trecho justificador: ”Os alunos reclamam que só descobrem o cancelamento quando chegam na sala”.

Prioridade: Média, é bom o aluno saber quando seu horário for cancelado, mas poderá checar manualmente para ter certeza.

Relatório de execução

Estrutura de Arquivos:

- classes.py: Contém a definição das entidades do sistema (Monitor, Aluno, Reserva, HorarioMonitoria e Disciplina).
- servicos.py: Contém a classe ServicoMonitoria, responsável pelas regras de negócio (listagem de horários disponíveis e realização de reservas).
- main.py: Arquivo principal com um script de demonstração e testes manuais básicos do fluxo do sistema.
- testesgreen.py: Suíte de testes unitários automatizados utilizando a bib unittest para garantir que as validações funcionem corretamente.

Como Executar:

O projeto não possui dependências externas. Primeiro executar a demonstração principal (main.py). Para rodar a simulação e ver o funcionamento básico no terminal: ***python main.py***

Em seguida, executar os Testes Automatizados (testesgreen.py): Para rodar todos os testes unitários e verificar o correto funcionamento das regras do sistema: ***python3 -m unittest -v testesgreen.py***

Comparação entre o Planejado e o Realizado

- (16/10 - 19/10) - Histórias de Usuário e Projeto Inicial: Responsáveis: Thiago (HUs) e Gabriel (Projeto). Concluído conforme o planejado.
- (19/10 - 21/10) - Implementação das Classes e Serviços: Responsáveis: Gabriel e Thiago (Em conjunto). Concluído em 20/10.
- (21/10 - 23/10) - Testes Automatizados e Refatoração: Responsáveis: Gabriel (Refatorar) e Thiago (Testes). Ajuste de Cronograma. Devido à apresentação presencial de ambos no SICITE em 21/10, a finalização dos testes (testesgreen.py) foi concluída em 22/10.

- (23/10 - 27/10) - Validação, Documentação Final e Envio: Responsáveis: Gabriel e Thiago. Concluido em 27/10.

Justificativa de Mudanças e Atrasos

1. Compromisso Acadêmico (SICITE): No dia 21/10/2026, ambos os integrantes participaram presencialmente das apresentações de trabalhos no SICITE. Por conta disso, foi solicitado adiamento para o envio final do relatório e o polimento de testes de aceitação foram deslocados para o dia 22/10, mantendo no prazo novo dado pelo professor de 29/10/2026.

2. Evolução do Projeto Inicial: Durante o desenvolvimento, optou-se por encapsular o método adicionar_horario() diretamente na classe Disciplina para evitar duplicação de horários no mesmo dia/hora, garantindo a coesão da entidade antes das chamadas no serviço.